

CREAR UN PROYECTO EN NODE Y EXPRESS

1. Abrir proyecto	1
2. Creamos el archivo .json	2
3. Preparamos el escritorio donde trabajar	3
3.1 Importamos las herramientas de desarrollo	3
3.2 Creamos rutas hacia nuestro archivo de datos	3
4. Empezamos a trabajar en las funciones. GET	4
5. Creamos servidor con su primera ruta GET	5

1. Abrir proyecto

1. Abrir carpeta del proyecto. Nombre proyecto.
2. Dentro de esta carpeta abrir dos carpetas: backend y frontend

Y dentro de la carpeta backend.

Inicializar un proyecto con Node

```
npm init -y
```

Instalar Express con sus dependencias básicas (middlewares)

```
npm i express cors morgan helmet compression
```

Para usar import

☐ Añadir "type": "module" en package.json

Instalar nodemon

```
npm install -D nodemon
```

Añadir los scripts de ejecución. dejar start y añadir dev

```
"scripts":  
{  
  "dev": "nodemon servidor.js" }  
}
```

Comenzamos a crear el proyecto.

En este proyecto vamos a usar dos archivos un archivo .json en el que crearemos algunos registros de datos para poderlos leer.

2. Creamos el archivo .json

Un archivo.json se crea abriendo un array [y dentro los elementos se separan entre llaves en formato clave: valor y con comas menos el último que no se separa con comas.

Un pequeño ejemplo

```
[  
  {  
    "id": 1,  
    "titulo": "La Llorona",  
    "artista": "Manu Chao",  
    "año": 2001  
  },  
  {  
    "id": 2,  
    "titulo": "Mediterráneo",  
    "artista": "Joan Manuel Serrat",  
  }  
]
```

```

    "año": 1971
  },
  {
    "id": 3,
    "titulo": "Se nos rompió el amor",
    "artista": "Rocío Jurado",
    "año": 1993
  }
]

```

Podrías crearlo tambien vacío si así cuadra con tu proyecto.

3. Preparamos el escritorio donde trabajar

3.1 Importamos las herramientas de desarrollo

```

// Express: Es como el "motor" de nuestro servidor web
import express from 'express';

// fs (File System): Nos permite leer y escribir archivos
import fs from 'fs';

// path: Nos ayuda a manejar rutas de archivos de forma correcta
import path from 'path';

// cors: Permite que nuestro frontend se comuniquen con el backend
import cors from 'cors';

// fileURLToPath: Necesario para obtener la carpeta actual en módulos
ES6
import { fileURLToPath } from 'url';

```

3.2 Creamos rutas hacia nuestro archivo de datos

```
// fileURLToPath: Necesario para obtener la carpeta actual en módulos
// ES6
import { fileURLToPath } from 'url';
// Obtener la ruta de la carpeta actual (equivalente a __dirname)
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const ARCHIVO_SENDEROS = path.join(__dirname, 'senderos.json');
```

3.3 Abrimos aplicación express

Creamos una variable en la que llamamos a express y en ella añadimos las utilidades que queremos usar

```
// Crear la aplicación de Express

const app = express();
app.use(cors());
app.use(express.json()); // Para entender JSON en las solicitudes
app.use(express.static(path.join(__dirname, './frontend')));
```

4. Empezamos a trabajar en las funciones. GET

```
function leerSenderos() {
  const contenido = fs.readFileSync(ARCHIVO_SENDEROS, 'utf-8');
  const senderos = JSON.parse(contenido);
  console.log(`📖 Se leyeron ${senderos.length} senderos del archivo`);
  return senderos;
}
```

```
}
```

console.log envia los mensajes de retroalimentación a la consola de visual code.

5. Creamos primeras rutas y levantamos servidor

```
const PORT=3001;

6.
7.
8. //creamos ruta
9. app.get('/api/senderos', (peticion, respuesta) => {
10.   console.log('💡 Solicitud: Dame todas las rutas de senderos!');
11.
12.   // Leer los senderos del archivo
13.   const senderos = leerSenderos();
14.
15.   // Enviar los senderos como respuesta
16.   respuesta.json(senderos);
17. });
18.
19. app.listen(PORT, () => {
20.   console.log(` Servidor escuchando en http://localhost:${PORT} `);
21. });
22.
```

5. Creamos servidor con su primera ruta GET

```
//creamos ruta
app.get('/api/senderos', (peticion, respuesta) => {
  console.log('💡 Solicitud: Dame todas las rutas de senderos!');

  // Leer los senderos del archivo
  const senderos = leerSenderos();

  // Enviar los senderos como respuesta
```

```
    respuesta.json(senderos);  
  });  
  
  app.listen(PORT, () => {  
    console.log(` Servidor escuchando en http://localhost:${PORT} `);  
  });
```

Este servidor tiene una sola ruta api/senderos

¿Por que se pone **api** en esta ruta? Realmente podrías llamarlo como quieras. Api indica que la ruta pertenece a la API (backend). Se entiende por API el enlace que se crea entre el front y el back para servir datos. La API: recibe una petición, la pasa al servidor, y devuelve una respuesta (por ejemplo, los datos que pediste). API significa "Interfaz de Programación de Aplicaciones": interfaz => punto de contacto. Programación => entre programas Aplicaciones=> referente al software puede ser una web, un servidor..

6. Función POST (crear nuevo)